

# Dify Internals: What Runs When You Press Go

A developer's guide to every Docker container in Dify 1.13.0 and why each one exists in an Agentic AI system.

- ❏ You don't start one app. You start a coordinated team of 12 specialised services.

DIFY V1.13.0

DEVELOPER TRAINING



# Why Not Just One Server?

Think of a hospital. One doctor cannot do surgery, run the pharmacy, manage records, guard the entrance, and schedule appointments simultaneously. Each role is specialised. Each has its own space. Each can fail without shutting down the others.

Dify works the same way. Twelve containers. Twelve specialised jobs. One agentic AI platform.

❏ **Key takeaway:** Separation of concerns makes the system safer, faster, independently scalable, and easier to debug.



# What You See When Dify Starts

The full output of `docker compose up -d` lists twelve entries. One is an init container that exits immediately. The other eleven run continuously.

## Init (exits)

`docker-init_permissions-1` — runs once, then exits cleanly

## Entry & UI

`docker-nginx-1`,  
`docker-web-1` — traffic routing and visual console

## Intelligence

`api`, `worker`,  
`worker_beat`,  
`plugin_daemon` — the thinking layer

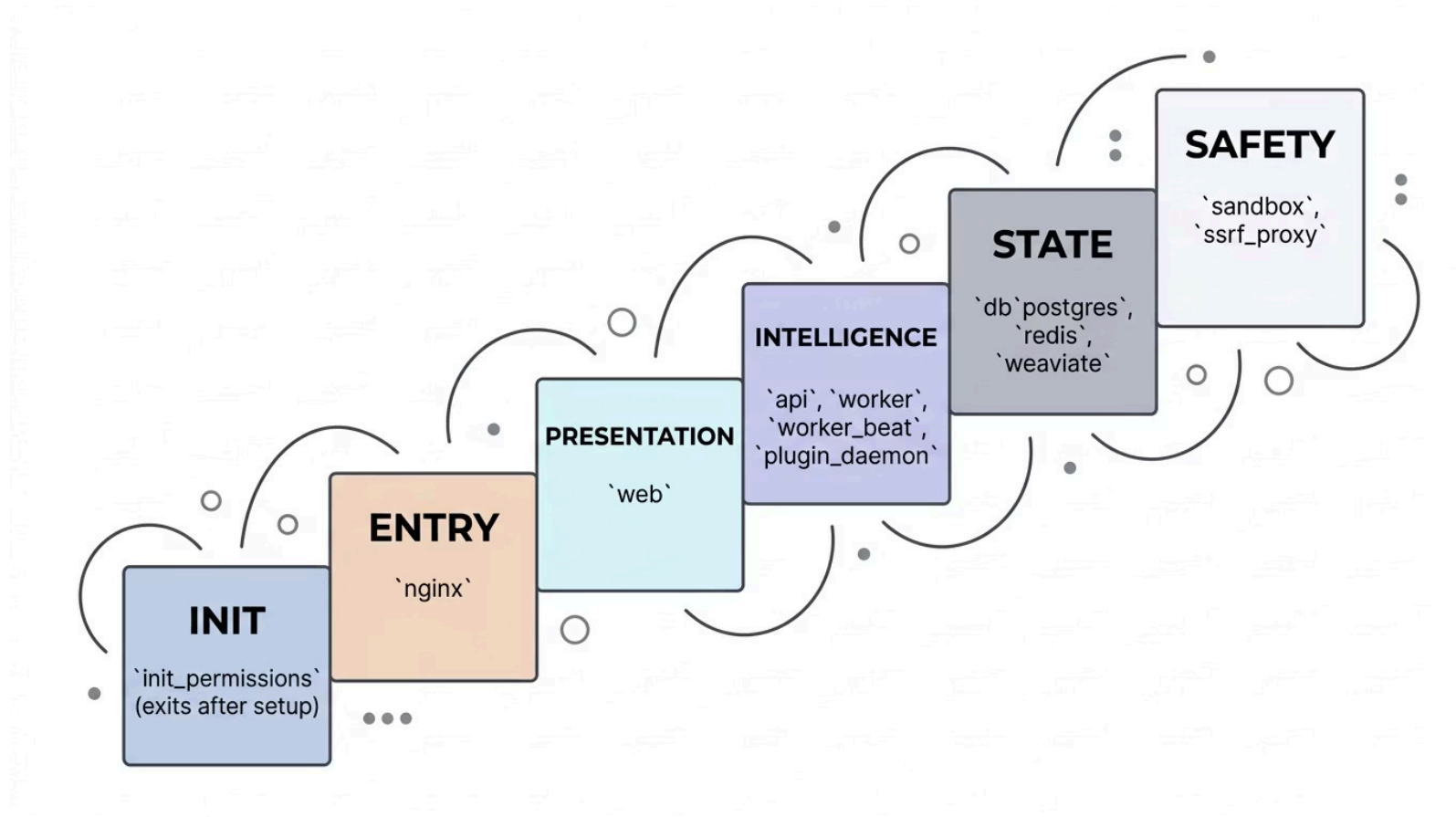
## State & Safety

`db`, `redis`, `weaviate`, `sandbox`, `ssrf_proxy`



# The Complete Container Map

Twelve containers organised into five distinct architectural layers. Three containers — `worker_beat`, `plugin_daemon`, and `init_permissions` — are new in Dify 1.x and each carries a specific, non-negotiable role.



# Layer Responsibilities at a Glance

Every layer owns a clearly defined slice of responsibility. No layer duplicates another's role — this is the architectural contract that makes Dify reliable.

| Layer        | Containers                              | Job                                               |
|--------------|-----------------------------------------|---------------------------------------------------|
| Init         | init_permissions                        | One-time setup before anything else runs          |
| Entry        | nginx                                   | Controls all traffic in and out                   |
| Presentation | web                                     | The visual canvas builders use                    |
| Intelligence | api, worker, worker_beat, plugin_daemon | Thinking, background work, scheduling, extensions |
| State        | db, redis, weaviate                     | All memory — permanent, fast, semantic            |
| Safety       | sandbox, ssrf_proxy                     | Code isolation and network protection             |

# How to Verify All Containers

## Your Daily Health Check

Run `docker compose ps` to see the live status of every container. One status is intentionally different from all the others.

### What to Look For

- `init_permissions` → **Exited (0)** — this is correct. Job done.
- All others → **Up** or **Healthy**

| Status     | Meaning                           |
|------------|-----------------------------------|
| Up         | Running normally                  |
| Healthy    | Running + passed health check     |
| Exited (0) | Finished successfully (init only) |
| Exited (1) | Crashed — check logs immediately  |
| Restarting | Crashing repeatedly — check logs  |



INIT LAYER

# Init Layer

A single, ephemeral container that prepares the environment before the platform starts. It runs once and exits — but without it, everything that follows breaks silently.

# init\_permissions: The Setup Crew

`docker-init_permissions` is a `busybox` container that runs a single `chown` command on shared storage directories, then exits cleanly with code 0.

## Why It Exists

Docker volumes are created by root by default. The `api`, `worker`, and `plugin_daemon` processes run as non-root users. Without this fix, they cannot write to storage directories — uploads fail silently and with no obvious error message.



## Expected Behaviour

**Exited (0)** — correct. Job is done.

**Exited (1)** — file permissions are not set. Other containers will fail on all file operations. Check logs immediately.

*Analogy: The site crew that unlocks all the doors and sets up access passes before the workforce arrives. Once done, they leave.*



ENTRY LAYER

# Entry Layer

Every request — whether from a browser, a developer API call, or a deployed agent integration — enters through a single controlled gateway. No container is directly reachable from outside.



# Nginx: The Front Door

`nginx` is the only container exposed to your network. It is a reverse proxy — the single entry point to the entire Dify platform. Every request arrives here first.

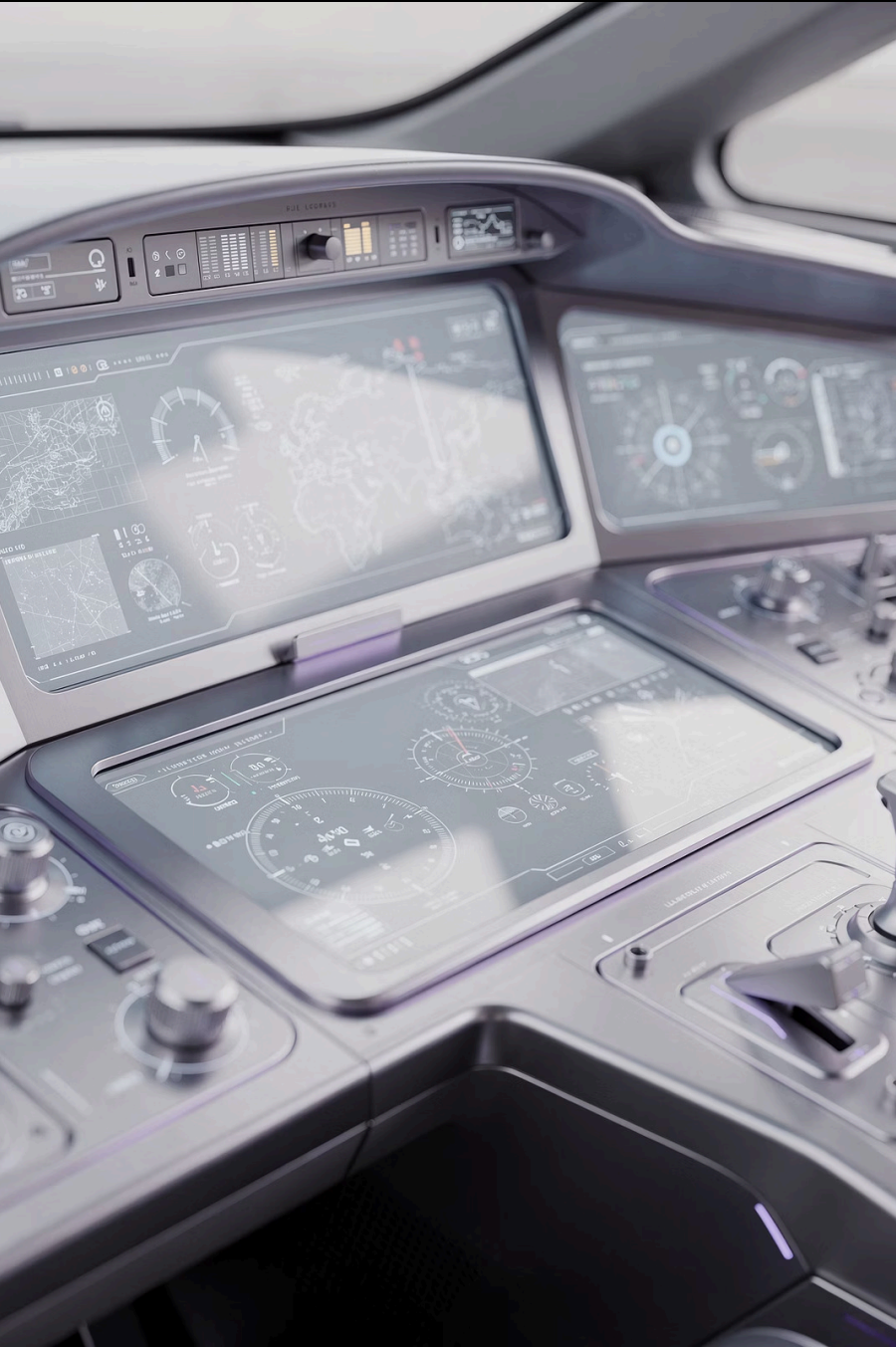
## What It Does

- Routes `/v1/*` and `/api/*` → `api` container
- Routes everything else → `web` container
- Handles HTTPS so other containers need not
- Holds connections open for long agentic runs

## Critical Agentic AI Setting

Nginx waits up to **3,600 seconds** for a response. A ReAct agent making 8 sequential tool calls can take several minutes. Default servers kill that connection at 60 seconds — nginx does not.

*Analogy: Reception at a large office. Every visitor goes through reception. No visitor walks directly into the server room.*



PRESENTATION LAYER

# Presentation Layer

The visual surface of Dify — where developers design workflows, test prompts, manage knowledge bases, and monitor agent behaviour. Pure interface; zero business logic lives here.

# Web: The Visual Builder

diffy-web is a Next.js frontend. It is the canvas where you design agents, build workflows, and monitor executions — but every action it shows is an API call to `diffy-api` under the hood.



## Workflow Canvas

Visual drag-and-drop builder for agent pipelines and DAG workflows



## Prompt Lab

Test and iterate on prompts, system instructions, and model parameters



## Knowledge Base UI

Upload, manage, and monitor document indexing for RAG pipelines



## Logs & Traces

Inspect every agent reasoning step, tool call, and token consumed



**Developer note:** Web going down = console inaccessible. Deployed agent APIs keep running. It is a developer outage, not a user outage.



INTELLIGENCE LAYER

# Intelligence Layer

Four containers that collectively think, process, schedule, and extend. This is the operational core of Dify — where agent reasoning loops execute, documents are indexed, tasks are scheduled, and plugins run in isolation.





# API: The Brain

`dify-api` is the agentic orchestrator — the only container that touches every other container. It does not store data, serve a UI, or run background jobs. It coordinates everything to produce a single intelligent response.

## What It Does

- Runs the agent ReAct loop end-to-end
- Executes workflow DAGs node by node
- Manages the full RAG pipeline
- Injects conversation memory into every LLM prompt
- Streams token-by-token responses back to the client

## Connections

- `redis` — task queue and cache
- `db` — agent config and history
- `weaviate` — semantic search
- `sandbox` — code execution
- External LLM APIs via `ssrf_proxy`

📌 **Warning:** If `api` goes down, everything stops. It is the only container with no graceful degradation.

```

graph LR
    A((?)) --> B((🔍📄))
    B --> C((🕒📖))
    C --> D((💡📖))
    D --> E((⬇️))
  
```

User Query      Load Instructions      Fetch History      Search KB      Build Prompt

Every iteration of the loop is fully logged to `db` in the `message_agent_thoughts` table — giving you complete auditability of why the agent took each action.



# Worker: The Background Processor

`dify-worker` handles tasks that are too long or too heavy to complete within a synchronous HTTP request. It reads jobs from the Redis queue and processes them asynchronously, without blocking the `api`.

## Document Indexing Pipeline



## Agentic AI Role

Every document your agent retrieves was built by this container. Worker creates and maintains the entire knowledge base that powers RAG.

*Analogy: Back-office team at a bank. The teller (api) serves customers at the window. Back office (worker) processes loan applications quietly without interrupting the teller.*

# worker\_beat: The Scheduler

worker\_beat is Celery Beat — a scheduler that fires periodic background tasks on a timetable. It triggers jobs; worker executes them. The distinction is critical.

## Example Schedule

- **Midnight daily:** clean up old logs
- **Every hour:** aggregate token usage stats
- **Every 10 minutes:** check model provider health

📌 **Why exactly one instance?** Two Beat schedulers would fire duplicate jobs — double cleanup, double stats, double notifications.

| worker                      | worker_beat                 |
|-----------------------------|-----------------------------|
| Reacts to jobs in the queue | Triggers jobs on a schedule |
| Scale to many replicas      | Always exactly ONE instance |
| On-demand                   | Time-driven                 |

*If worker\_beat goes down: scheduled maintenance stops. Logs grow unbounded. Usage stats freeze. No immediate user impact, but platform degrades over days.*



# plugin\_daemon: The Extension Runtime

`dify-plugin_daemon` is a dedicated runtime for Dify plugins, introduced in Dify 1.0. Before 1.0, extending Dify meant modifying core code. Now plugins are installable packages running in isolation — a crashing plugin cannot crash your `api`.

## New LLM Providers

Add model providers not natively supported by Dify core

## Tool Integrations

Connect Slack, Jira, custom APIs, and proprietary systems as agent tools

## Document Processors

Custom parsers for PDFs, spreadsheets, and proprietary file formats

## Workflow Nodes

Add new node types to the workflow canvas for specialised operations

📌 *Analogy: The app store runtime on your phone. The phone (api) keeps working even if one app (plugin) crashes. Your custom banking tool integration lives here.*

# The Intelligence Layer: All Four Together

The four intelligence containers divide all work into two fundamental modes: real-time (synchronous, user-facing) and background (asynchronous, deferred). Every task in Dify belongs to exactly one mode.

## Real-Time (Synchronous)

- **api** — serves user requests now, runs the agent loop live
- **plugin\_daemon** — handles plugin calls that api makes inline

## Background (Asynchronous)

- **worker** — processes jobs from the Redis queue at its own pace
- **worker\_beat** — fires scheduled recurring jobs on a timetable

📌 **Developer rule:** NOW → api. LATER → worker. REGULARLY → worker\_beat. PLUGIN CAPABILITY → plugin\_daemon.





STATE LAYER

# State Layer

Three containers that collectively hold every piece of memory the platform needs — permanent and relational, fast and ephemeral, and semantic and vectorised. Lose the state layer and you lose the platform.

# Database: Permanent Memory

dify-db (PostgreSQL) is the source of truth for everything in the platform. It is the only container whose data must be treated as irreplaceable.

## What It Stores

- Every app and workflow ever built
- Every conversation ever held
- Every agent reasoning step
- Every document chunk (text)
- Every execution log
- Every plugin configuration

## Key Boundary

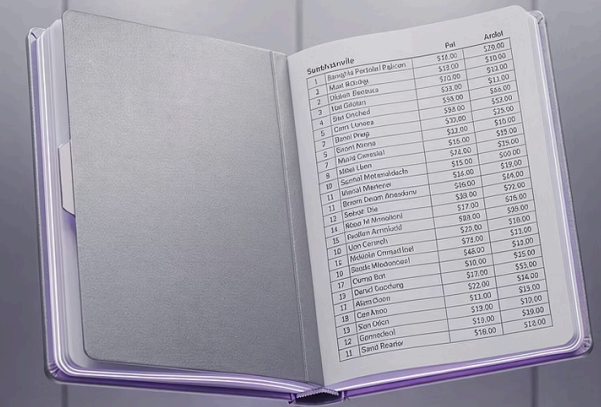
Postgres stores chunk **TEXT**.

Weaviate stores chunk **VECTORS**.

They are linked by chunk ID.

Neither works without the other.

❏ **Critical:** If db goes down, api and worker crash immediately. Total outage. Back this up daily.



# The Key Tables for Agent Behaviour

Six tables in PostgreSQL directly govern what your agent does, what it remembers, and how it reasons at runtime. Understanding these tables is essential for debugging agent behaviour.

| Table                    | Question it answers                  |
|--------------------------|--------------------------------------|
| apps                     | How should this agent behave?        |
| messages                 | What did we discuss before?          |
| message_agent_thoughts   | Why did the agent take that action?  |
| dataset_segments         | What does the agent know?            |
| workflow_node_executions | What happened at each workflow step? |
| plugin_configs           | Which plugins are installed and how? |

# Redis: Three Jobs, One Container

`dify-redis` serves three entirely distinct purposes simultaneously. It is the high-speed layer that prevents the slower PostgreSQL database from becoming a bottleneck on every request.

## Job Queue

Passes jobs between `api` → `worker` and `worker_beat` → `worker`. The reliable handoff mechanism for all background work.

## Application Cache

Hot read data — API key validation, app config, rate limits — answered in microseconds, not milliseconds. Reduces db load dramatically.

## Streaming State

Tracks live streaming agent runs in progress. Stores working memory for mid-execution workflow state.

- ❏ **Production warning:** Redis stores in RAM only by default. A restart means queued jobs are lost. Enable AOF persistence for any production deployment.



# Weaviate: The Semantic Search Engine

`dify-weaviate` is the vector database that makes RAG (Retrieval-Augmented Generation) possible. It stores numerical representations of meaning — embeddings — and finds the most semantically similar content for any query in milliseconds.

## What Is an Embedding?

Every piece of text is converted to a vector — 1,500+ numbers that encode its meaning. Similar meaning = similar numbers. Weaviate searches these numbers at scale.

"What is the RBI penalty for late filing?"

↓ Embedding model

[0.023, -0.147, 0.891, 0.034 ...]

## The RAG Flow

- User query → converted to numbers
- Weaviate searches all stored vectors
- Returns top 5 most similar chunk IDs
- Postgres fetches actual text for those IDs
- Text injected into agent's LLM prompt

❏ **Without Weaviate:** agents cannot search your documents — they answer only from LLM training data.

# Weaviate and Postgres: Always a Pair

Weaviate and PostgreSQL are permanently coupled through chunk IDs. They divide the labour of semantic retrieval — one finds relevance, the other delivers content. Neither is useful without the other.

## Weaviate

**Stores:** Vectors (the numbers)

**Good at:** Finding similar items fast

**Answers:** WHICH chunks are relevant?

## Postgres

**Stores:** Chunk text (the words)

**Good at:** Fetching text by exact ID

**Answers:** WHAT do those chunks say?

- ❏ **Critical rule:** Never delete one volume without re-indexing from the other. They must always stay in sync — they reference each other by chunk ID.



SAFETY LAYER

# Safety Layer

Two containers whose entire purpose is to contain risk — one isolates code execution, the other controls outbound network access. In a production agentic system, both are non-negotiable.



# Sandbox: The Code Jail

`dify-sandbox` executes untrusted code from workflow Code Nodes in strict isolation. The `api` sends code over HTTP; only the output comes back. Nothing dangerous can escape the container boundary.

## Isolation Constraints

- × No internet access
- × No database access
- × No file system writes
- × No spawning other processes
- ✓ Returns output only

## Agentic AI Role

Code Nodes do what LLMs do unreliably — exact arithmetic, data transformation, API response parsing. Sandbox makes this production-safe by ensuring no side effects escape.

*Analogy: A quarantine room. Tests run inside. Only results come out. Nothing dangerous can escape.*

# ssrf\_proxy: The Outbound Firewall

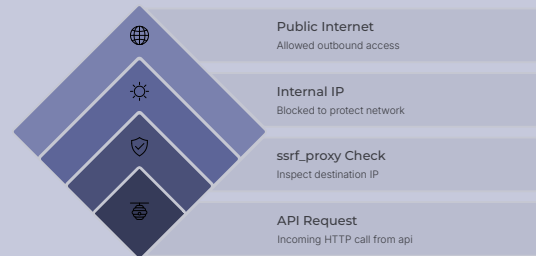
`dify-ssrf_proxy` protects your internal network from Server-Side Request Forgery attacks — where a maliciously crafted workflow HTTP Node could instruct `dify-api` to call your internal systems directly.

## The Threat

Without protection, an HTTP Node configured with these URLs causes `dify-api` to make the request:

- `http://192.168.1.1/admin` (internal router)
- `http://10.0.0.5/corebanking` (internal system)
- `http://169.254.169.254/` (cloud metadata = credentials)

## The Solution



*Banking importance: Your core banking systems run on private IPs. The proxy ensures agent tool calls can never reach them accidentally or maliciously.*



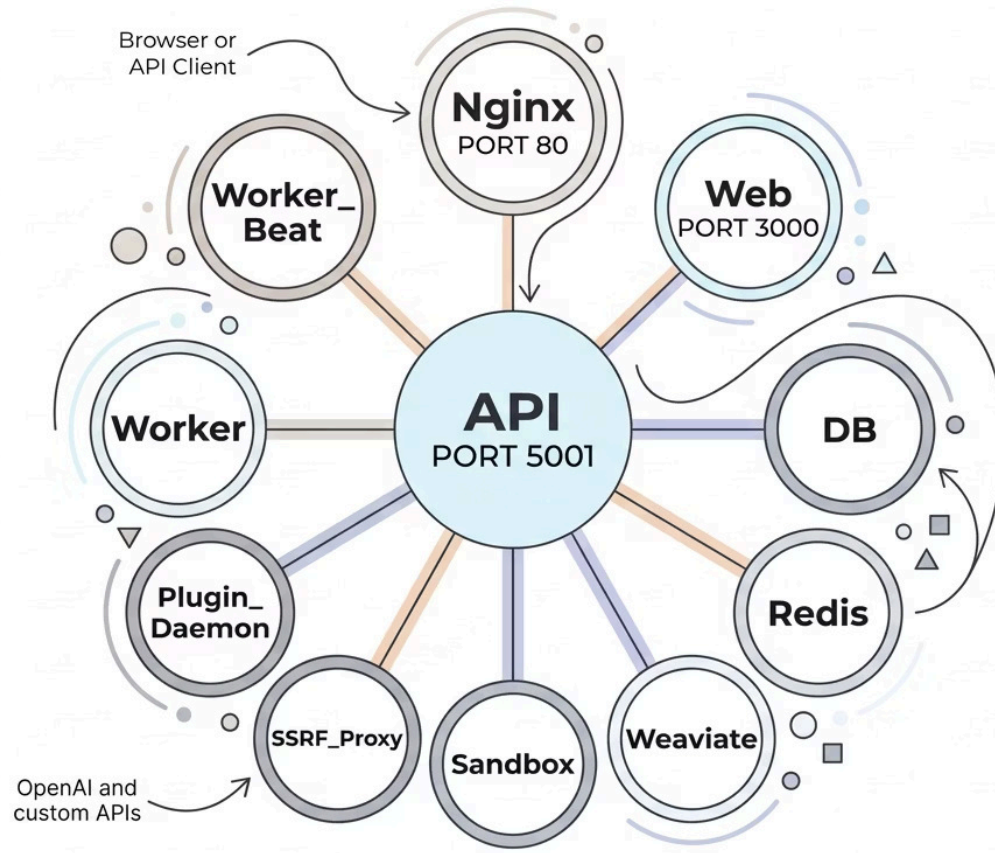
HOW IT ALL WORKS TOGETHER

# How It All Works Together

Eleven containers running simultaneously, each with a defined communication contract. Understanding the full map tells you exactly where to look when something goes wrong.

# The Full Communication Map

Only `nginx` port 80 is exposed to your network. Everything else communicates on Docker's internal network only — invisible and unreachable from outside the host.



# Startup Order

Docker Compose respects dependency declarations — but the ordering also reflects your recovery strategy after an outage. Always restart from the bottom up.



## Phase 1 — No Dependencies

Start immediately: `db` → `redis` → `weaviate` → `sandbox` → `ssrf_proxy`



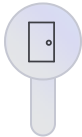
## Phase 2 — Needs Phase 1 Healthy

`api` → `worker` → `worker_beat` → `plugin_daemon`



## Phase 3 — Needs Phase 2 Healthy

`web` starts only after `api` is confirmed running



## Phase 4 — Needs Everything Healthy

`nginx` opens the door only when all containers behind it are ready



**Most common first-run problem:** `api` crashes because Postgres is still initialising. Fix: `docker compose restart api` after 30 seconds.



# Agent Memory Across Containers

An agent's "memory" is not stored in one place. It is distributed across three containers, each holding a different type of knowledge. Understanding this map tells you exactly what breaks when a container fails.

| Memory type         | Container     | What is stored                      | Failure symptom                   |
|---------------------|---------------|-------------------------------------|-----------------------------------|
| Instructions        | db            | Agent persona, tools, system prompt | Forgets everything, uses defaults |
| Conversation        | db            | Full message history                | No context between turns          |
| Semantic knowledge  | weaviate + db | Vectors + chunk text                | Cannot find documents             |
| Working state       | redis         | Mid-run workflow values             | Workflow breaks mid-run           |
| Plugin capabilities | plugin_daemon | Plugin state and config             | Plugin-based tools fail silently  |

# Data Persistence Overview

Not all containers persist data equally. Knowing which containers are stateful and which are stateless is critical for backups, restarts, and disaster recovery planning.

## Always Persisted (Disk Volume)

`db` and `weaviate` — survive every restart. Must be backed up together. Never delete `db_data` or `weaviate_data` independently.

## Persisted with Configuration

`redis` — stores in RAM by default. Enable AOF (Append-Only File) persistence for production. Without it, a restart loses all queued jobs.

## Stateless (Always Safe to Restart)

`api`, `worker`, `worker_beat` — hold no state themselves. In-flight jobs in worker may be lost on restart; all other state lives in the state layer.



# When Things Go Wrong

Every symptom in Dify points to a specific container. This is your systematic triage guide — always start with `docker compose ps`, then drill into `docker compose logs [container-name]`.

| What you see                | First check   | Second check          |
|-----------------------------|---------------|-----------------------|
| Console not loading         | nginx         | web                   |
| API calls returning errors  | api           | db or redis           |
| Document stuck "Indexing"   | worker        | Redis queue depth     |
| Agent says "I don't know"   | weaviate      | Indexing pipeline     |
| Code Node failing           | sandbox       | Sandbox logs directly |
| HTTP Node failing           | ssrf_proxy    | Is the URL blocked?   |
| Plugin tool not working     | plugin_daemon | Plugin install status |
| Scheduled tasks not running | worker_beat   | Redis connectivity    |
| Everything down             | db            | Then redis            |

# Summary: All 12 at a Glance

Twelve containers, five layers, one agentic AI platform. Every container has a single job and a clear role in making your agents think, remember, and act safely.

| Container        | Layer        | One Job                           | Agentic AI Role                             |
|------------------|--------------|-----------------------------------|---------------------------------------------|
| init_permissions | Init         | Fix volume permissions at startup | Enables safe file operations                |
| nginx            | Entry        | Route all traffic                 | Holds long agent connections open           |
| web              | Presentation | Visual workflow builder           | Design agent pipelines visually             |
| api              | Intelligence | Orchestrate everything            | Runs the agent reasoning loop               |
| worker           | Intelligence | Background job processing         | Builds agent knowledge (indexes docs)       |
| worker_beat      | Intelligence | Schedule recurring tasks          | Keeps platform healthy over time            |
| plugin_daemon    | Intelligence | Plugin runtime                    | Extends agent capabilities safely           |
| db               | State        | Permanent storage                 | Agent long-term memory                      |
| redis            | State        | Fast cache and queue              | Agent working memory + task handoff         |
| weaviate         | State        | Vector search                     | Semantic document retrieval                 |
| sandbox          | Safety       | Isolated code execution           | Safe deterministic computation              |
| ssrf_proxy       | Safety       | Outbound HTTP firewall            | Keeps tool calls away from internal systems |

❏ **One sentence:** init sets up → nginx routes in → web lets you build → api runs agents → worker/beat/plugin extend capabilities → db/redis/weaviate remember everything → sandbox/ssrf keep it safe.